

Python 基础

魏舟

2026 年 1 月 20 日

目录

1 前言

这是关于 Python 基础用法笔记，结合杜珊珊老师上课所讲和后期整理。

2 语言特点

- 解释型语言：编译一句，运行一句（如 Python, JavaScript）。
- 编译型语言：全编译完了进行运行（如 C, C++）。
- 动态类型：变量不需要显式声明类型，类型在运行时确定。
- 强类型：不同类型之间不会发生隐式的、意外的转换（如字符串和数字不能直接相加）。

3 环境配置与 pip 命令

pip 是 Python 的包管理工具。

- 常用命令：
 - 查看帮助信息：`pip --help`
 - 安装包：`pip install < 包名 >`
 - 指定版本安装：`pip install < 包名 >==1.2.3`
 - 镜像站加速安装：`pip install < 包名 > -i https://pypi.tuna.tsinghua.edu.cn/simple`
 - 查看已安装列表：`pip list`
 - 卸载包：`pip uninstall < 包名 >`
 - 导出依赖列表：`pip freeze > requirements.txt`

4 书写规则与规范

1. 语句分隔：一句一行；若一行多句，采用语句分隔符 `;`；若一句多行，结尾用 `\`。
2. 首行缩进：程序不能有无意义的空格，否则产生 `IndentationError`。
3. 符号状态：所有符号必须是 英文半角状态。
4. 缩进规范：推荐使用 4 个空格，**严禁**混合使用 Tab 和空格。遵循 PEP 8 风格指南。
5. 注释：
 - 单行：`#`
 - 多行/文档字符串：`''' 内容'''` 或 `""" 内容"""`

5 数据类型 (Data Types)

Python 不需要显式定义数据类型（动态类型）！ 数的范围只受计算机系统内存限制。

数据类型	名称	示例	说明
int	整数	1024, -5	无取值范围限制
float	浮点数	3.14, 2e-3	符合 IEEE 754 标准的双精度浮点数
str	字符串	'Hi', "Python"	不可变序列, 支持多行 ''' '''
bool	逻辑值	True, False	实际上是 int 的子类 (1 和 0)
complex	复数	4+7.6j	由实部和虚部组成
NoneType	空值	None	表示变量尚未赋值或无意义

5.1 基本数据类型

5.2 组合数据类型 (Container Types)

5.2.1 序列类共通操作 (list, tuple, str)

序列索引与切片: `sname[start : end : step]`

- 正向索引: 从 0 开始 (从左往右)。
- 反向索引: 从 -1 开始 (从右往左)。
- 切片原则: 包含 `start`, 不包含 `end` (左闭右开)。

内置函数:

- `len(s)`: 返回长度。
- `x in s`: 检查元素 `x` 是否在序列中。
- `s + t`: 连接两个序列。
- `s * n`: 将序列重复 `n` 次。

6 核心数据结构总结

6.0.1 列表 (list)

定义与构造

- 定义: 列表是 Python 中最灵活的序列类型。它是一个**有序**的容器, 支持**索引**访问, 且属于**可变对象 (Mutable)**。
- 构造方式:
 - 常用字面量: `ls = [10, 'AI', 3.14]`
 - 使用 `list()` 转换可迭代对象: `list("Hello") → ['H', 'e', 'l', 'l', 'o']`
 - 列表推导式 (List Comprehension): `[x*2 for x in range(3)] → [0, 2, 4]`

索引与切片 (Indexing & Slicing)

- 正逆索引: `L[0]` 访问首元素, `L[-1]` 访问末尾元素。

- 切片语法: `L[start:stop:step]`
 - `L[1:4]`: 获取索引 1, 2, 3 的元素 (左闭右开)。
 - `L[::-1]`: 将列表翻转。
 - `L[:]`: 生成列表的一个浅拷贝 (Shallow Copy)。

核心操作方法

类别	方法/函数	功能描述与示例
添加	<code>append(x)</code>	在末尾添加元素: <code>[1].append(2) → [1, 2]</code>
	<code>insert(i, x)</code>	在索引 <i>i</i> 处插入 <i>x</i> : <code>[1, 3].insert(1, 2) → [1, 2, 3]</code>
	<code>extend(L)</code>	拼接列表: <code>[1].extend([2, 3]) → [1, 2, 3]</code>
删除	<code>pop([i])</code>	默认删除末尾 (或指定索引) 并返回该值
	<code>remove(x)</code>	删除第一个值为 <i>x</i> 的元素, 若不存在则报 <code>ValueError</code>
	<code>clear()</code>	清空列表内容
排序	<code>sort()</code>	原地升序排序; <code>sort(reverse=True)</code> 为降序
	<code>reverse()</code>	原地反转列表顺序
统计	<code>count(x)</code>	统计 <i>x</i> 出现的次数
	<code>index(x)</code>	查找 <i>x</i> 第一次出现的索引位置

常用内置函数 (Built-in Functions) 针对列表作为参数的常用函数:

- `len(L)`: 返回列表长度。
- `max(L)` / `min(L)`: 返回列表中的最大/最小值 (要求元素类型可比较)。
- `sum(L)`: 对数值型列表求和。
- `sorted(L)`: 返回新列表, 原列表顺序不变 (注意与 `L.sort()` 的区别)。
- `enumerate(L)`: 在循环中同时获取索引和值: `for i, v in enumerate(['a', 'b']): ...`

嵌套列表 (Nested Lists) 列表可以包含其他列表, 用于表示矩阵或多维数据:

- 示例: `matrix = [[1, 2], [3, 4]]`
- 访问: `matrix[1][0]` 将得到 3。

注意事项:

列表是引用传递。当执行 `list_b = list_a` 时, 两者指向同一内存地址。若需独立副本, 请使用 `list_a.copy()` 或 `list_a[:]`。

```

1 # 列表推导式与常用操作
2 nums = [1, 2, 3]
3 nums.append(4)                # [1, 2, 3, 4]
4 squares = [x**2 for x in nums if x % 2 == 0] # [4, 16] (带过滤的推导式)
5

```

```

6 # 列表切片高级用法
7 rev_nums = nums[::-1]          # 快速反转副本

```

Listing 1: 列表高级操作

6.0.2 元组 (tuple)

定义与构造

- **定义：**元组是一个**有序**的序列，一旦创建便**不可修改 (Immutable)**。它通常用于存储异构数据（即不同类型的数据），类似于数据库中的一条记录。
- **构造方式：**
 - **字面量：** `t = (1, 'Python', 3.14)`
 - **单元素元组：**必须包含逗号，否则会被视为括号运算符。示例：`t = (1,)` 是元组，而 `t = (1)` 是整数。
 - **隐式构造：** `x = 1, 2, 3` (逗号才是构建元组的关键)。
 - **转换：** `tuple([1, 2]) → (1, 2)`。

为什么需要元组？（元组 vs 列表）

- **安全性：**数据不可变，防止程序意外修改，适用于常量配置。
- **性能：**元组比列表更轻量，由于大小固定，遍历速度更快，内存占用更小。
- **作为键使用：**元组是可哈希的 (Hashable)，因此可以作为字典的 **key** 或集合 (**set**) 的元素，而列表不行。

核心操作与特性

类别	操作示例	功能描述
基础操作	<code>t1 + t2</code>	拼接两个元组，生成一个新元组。
	<code>t * n</code>	重复元组 n 次。
统计方法	<code>t.count(x)</code>	返回元素 x 出现的次数。
	<code>t.index(x)</code>	返回元素 x 第一次出现的索引。
内置函数	<code>len(t), max(t), min(t)</code>	与列表一致，获取长度或极值。
	<code>sum(t)</code>	对数值元组求和。

进阶用法：解包 (Unpacking) 元组最强大的功能之一是解包赋值，这在 Python 函数返回多个值时非常常见：

- **基本解包：** `a, b = (10, 20) → a=10, b=20`。
- **交换变量：** `x, y = y, x` (无需临时变量)。

- 星号解包 (Python 3.5+):

- first, *middle, last = (1, 2, 3, 4, 5)
- 结果: first=1, middle=[2, 3, 4], last=5。

具名元组 (namedtuple) 为了提高代码可读性, 可以使用 collections 模块中的 namedtuple。

- 示例:

```
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(11, 22)
print(p.x) # 输出 11, 可以通过属性名访问
```

注意事项:

不可变的相对性: 如果元组内部包含可变对象 (如列表), 则该列表的内容是可以修改的。示例: `t = (1, [2, 3])`, 执行 `t[1][0] = 99` 是合法的, 因为元组只保证其持有的“引用”不发生变化。

6.0.3 字典 (dict)

定义与核心特性

- 本质: 字典是 Python 中唯一的映射类型 (Mapping Type), 存储的是键值对 (Key-Value pairs)。
- 性质: 可变、键唯一。从 Python 3.7+ 开始, 字典正式保证了元素的插入顺序。
- 哈希机制: 键必须是不可变对象 (如 `str`, `int`, `tuple`), 因为字典通过哈希算法实现 $O(1)$ 时间复杂度的极速查找。

多样化的构造方法

- 标准字面量: `d = {'id': 101, 'name': 'Gemini'}`
- 关键字参数: `d = dict(city='Beijing', zip=100000)`
- 元组列表转换: `d = dict([('a', 1), ('b', 2)])`
- 字典推导式: `d = {x: x**2 for x in range(5) if x % 2 == 0}`
结果: `{0: 0, 2: 4, 4: 16}`
- 空字典: `d = {}` 或 `d = dict()`

核心操作函数与详细示例

进阶操作与现代 Python 特性

- 成员检测: `'Alice' in d` (检测是否为字典的键)。

类别	方法 / 语法	功能描述与代码示例
存取	<code>d[key]</code>	取值。若键不存在会报 <code>KeyError</code> 。
	<code>d.get(k, v)</code>	安全取值。若键不存在则返回 <code>v</code> 。 示例: <code>d.get('age', 18)</code> → 返回 18(若无 age)
修改	<code>d[k] = v</code>	新增或覆盖。若键已存在则覆盖旧值。
	<code>d.update(new_d)</code>	批量合并。将 <code>new_d</code> 的内容合并至原字典。
删除	<code>pop(k)</code>	弹出并返回键 <code>k</code> 对应的值。
	<code>popitem()</code>	删除末尾。返回最后插入的 (key, value) 元组。
	<code>del d[k]</code>	关键字删除。彻底移除该条目。
视图	<code>d.keys()</code>	返回所有键的动态视图。
	<code>d.values()</code>	返回所有值的动态视图。
	<code>d.items()</code>	返回所有 (key, value) 元组。用于 for 循环。

- 字典合并 (Python 3.9+):

- `new_dict = d1 | d2` (返回合并后的新字典)
- `d1 |= d2` (原地更新 d1)

- 遍历技巧:

```
# 同时解包键与值
for k, v in d.items():
    print(f"Key: {k}, Value: {v}")
```

- 设置默认值: `d.setdefault('count', 0)`。若 'count' 不存在, 则将其设为 0 并返回 0; 若存在则返回已有值。

重点总结

注意 1: 不可变键限制

字典的键必须可哈希。因此: `{[1, 2]: 'val'}` 会抛出错误, 而 `{(1, 2): 'val'}` 是合法的。

注意 2: 浅拷贝问题

`d2 = d1.copy()` 仅复制父对象。如果字典内部嵌套了列表, 修改 `d2` 中的列表仍会影响 `d1`。

注意 3: 性能对比

列表的查找复杂度为 $O(n)$, 而字典为 $O(1)$ 。在处理海量数据去重或快速索引时, 字典是首选。

6.0.4 集合 (set)

定义与核心特性

- **本质**：集合是一个**无序、不重复**的元素序列。
- **性质**：**可变对象**（但集合内的元素必须是不可变的/可哈希的）、**唯一性**（自动去重）。
- **底层实现**：与字典类似，集合基于散列表（Hash Table）实现，因此 `in` 操作的成员检测速度极快，时间复杂度平均为 $O(1)$ 。

构造方法与示例

- **字面量（大括号）**：`s = {1, 2, 2, 3}` $\rightarrow$ `{1, 2, 3}`（重复元素自动消失）。
- **set 构造函数**：`s = set([1, 2, 3, 2])` $\rightarrow$ `{1, 2, 3}`。
- **构造空集合**：必须使用 `set()`。注意 `{}` 创建的是空字典。
- **集合推导式**：`s = {x for x in 'abracadabra' if x not in 'abc'}` $\rightarrow$ `{'r', 'd'}`。

—

核心操作函数与详细示例

类别	方法 / 语法	功能描述与代码示例
增加	<code>add(x)</code>	添加单个元素。若 x 已存在则无影响。
	<code>update(iter)</code>	批量添加。示例： <code>s.update([4, 5])</code> 。
删除	<code>remove(x)</code>	移除元素。若 x 不存在会报 <code>KeyError</code> 。
	<code>discard(x)</code>	安全移除。若 x 不存在则静默处理，不报错。
	<code>pop()</code>	随机弹出并返回一个元素。由于集合无序，弹出对象不确定。
清空	<code>clear()</code>	移除集合内所有元素。
	<code>len(s)</code>	返回集合中元素的个数。

—

强大的集合运算（数学运算） 集合最显著的用途是进行数学意义上的集合操作。假设 `a = {1, 2, 3}`, `b = {3, 4, 5}`：

- **并集 (Union)**：`a | b` 或 `a.union(b)` $\rightarrow$ `{1, 2, 3, 4, 5}`
- **交集 (Intersection)**：`a & b` 或 `a.intersection(b)` $\rightarrow$ `{3}`
- **差集 (Difference)**：`a - b` 或 `a.difference(b)` $\rightarrow$ `{1, 2}`
- **对称差集 (Symmetric Difference)**：`a ^ b`（只属于 `a` 或只属于 `b` 的元素） $\rightarrow$ `{1, 2, 4, 5}`

—

进阶判定与不可变集合

- 子集判定: `s1.issubset(s2)` 或 `s1 <= s2`。
- 父集判定: `s1.issuperset(s2)` 或 `s1 >= s2`。
- 无交集判定: `s1.isdisjoint(s2)` (若无共同元素返回 `True`)。
- `frozenset` (不可变集合):
 - 特性: 一旦创建不可修改, 因此是可哈希的。
 - 用途: 可以作为字典的键, 或者作为另一个集合的元素。
 - 示例: `fs = frozenset([1, 2, 3])`。

重点总结

最佳实践: 快速去重

将列表转换为集合再转回列表是 Python 中最常用的去重手段: `ls = list(set(ls))`。

注意: 集合的无序性

集合不支持索引访问 `s[0]`。如果需要按顺序处理, 必须先转换为列表或使用 `sorted(s)`。

注意: 成员检测性能

在包含 100 万个元素的容器中寻找一个值: 列表需要扫描整个容器 ($O(n)$), 而集合瞬间完成 ($O(1)$)。

7 程序控制流

7.1 分支结构 (Conditional Statements)

Python 使用缩进来界定代码块。

```
1 score = int(input("请输入分数: "))
2 if score >= 90:
3     print("优秀")
4 elif score >= 60:
5     print("及格")
6 else:
7     print("不及格")
```

7.2 循环结构 (Loops)

- **for 循环:** 迭代对象 (列表、范围、字符串等)。
- **while 循环:** 基于条件判断的循环。
- **控制关键字:** `break` (跳出循环)、`continue` (进入下一次迭代)。

```
1 # 遍历 range
2 for i in range(1, 10, 2): # start=1, stop=10, step=2
3     print(i) # 输出 1, 3, 5, 7, 9
4
```

```

5 # while 循环
6 count = 0
7 while count < 3:
8     print("Looping...")
9     count += 1

```

8 函数 (Functions)

函数是组织好的，可重复使用的，用来实现单一或相关联功能的代码段。它提高了应用的模块性以及代码的复用率。

8.1 函数定义与调用

```

1 def 函数名(形参列表):
2     """文档字符串 (Docstring), 用于解释函数功能"""
3     函数体...
4     return [返回值] # 若省略则默认返回 None

```

关键点：

- 形参 (Parameters)：定义函数时括号内的变量。
- 实参 (Arguments)：调用函数时传递给它的具体值。
- 位置参数：调用时必须按顺序传入，数量需与定义一致。
- 关键字参数：调用时使用 `name=value` 形式，此时顺序可以不固定。

```

1 def greet(name, msg="Good morning!"):
2     return f"Hello {name}, {msg}"
3
4 print(greet("Bob")) # 使用默认参数
5 print(greet(msg="Hi", name="Alice")) # 关键字参数，顺序可变

```

8.2 变长参数 (Variable-length Arguments)

当不确定调用者会传递多少个参数时，使用变长参数。

8.2.1 元组类型变长参数 (*args)

`*args` 用于接收任意数量的非关键字（位置）参数，并将其打包成一个元组。

- 规则：若函数定义中同时存在位置参数和 `*args`，`*args` 必须放在位置参数之后。

8.2.2 字典类型变长参数 (**kwargs)

****kwargs** 用于接收任意数量的关键字参数，并将其打包成一个字典。

```
1 def func(a, *args, **kwargs):
2     print(f"a: {a}")
3     print(f"args: {args}")    # 表现为元组
4     print(f"kwargs: {kwargs}") # 表现为字典
5
6 func(1, 2, 3, 4, name="Tech", age=20)
7 # 输出: a: 1, args: (2, 3, 4), kwargs: {'name': 'Tech', 'age': 20}
```

8.3 返回值 (Return Value)

- 单一返回值：直接返回该对象。
- 多个返回值：Python 允许函数返回多个值，实质上是将其打包成一个 **Tuple (元组)**。

```
1 def get_point():
2     return 10, 20 # 等同于 return (10, 20)
3
4 x, y = get_point() # 解包赋值
```

8.4 函数作用域 (Scope)

Python 遵循 **LEGB** 规则：Local (局部) → Enclosing (嵌套) → Global (全局) → Built-in (内建)。

- **global**：在函数内部修改全局变量。
- **nonlocal**：在嵌套函数中修改外层非全局变量。

```
1 def out_def():
2     x = 1 # 闭包变量
3     y = 2
4     def in_def():
5         nonlocal x
6         x = 100 # 修改了外层的 x
7         y = 200 # 仅在 in_def 局部创建了 y，不影响外层
8         print(f"in_def: x={x}, y={y}")
9
10    in_def()
11    print(f"out_def: x={x}, y={y}") # x 变为 100, y 仍为 2
12
13 out_def()
```

8.5 Lambda 匿名函数

Lambda 表达式用于创建简洁的、单行的匿名函数。常用于高阶函数如 `map()`, `filter()`, `sorted()`。

语法: `lambda 参数: 表达式`

```
1 # 基础用法
2 add = lambda a, b: a + b
3 print(add(3, 5)) # 8
4
5 # 在列表排序中的应用
6 points = [(1, 2), (4, 1), (5, -1)]
7 points.sort(key=lambda x: x[1]) # 按元组第二个元素排序
8 print(points) # [(5, -1), (4, 1), (1, 2)]
```

8.6 高阶函数 (Higher-order Functions)

函数在 Python 中是“一等公民”，可以作为参数传递，也可以作为返回值。

- `map(func, iterable)`: 对序列中的每个元素应用函数。
- `filter(func, iterable)`: 过滤掉不符合条件的元素。

```
1 nums = [1, 2, 3, 4]
2 squared = list(map(lambda x: x**2, nums)) # [1, 4, 9, 16]
3 evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
```

9 异常处理 (Exception Handling)

通过捕获异常可以增强程序的鲁棒性。

```
1 try:
2     <语块1>
3 except <异常类型1>:
4     <语块2>
5 except <异常类型2>:
6     <语块3>
7 else:
8     <语块4> # 没有发生异常执行
9 finally:
10    <语块5> # 无论是否发生异常都会执行
```

```
1 try:
2     num = int(input("输入除数: "))
3     result = 100 / num
4 except ZeroDivisionError:
5     print("错误: 除数不能为零")
6 except ValueError:
```

```

7     print("错误：请输入有效的整数")
8 except Exception as e:
9     print(f"未知错误：{e}")
10 else:
11     print(f"计算成功，结果是：{result}")
12 finally:
13     print("无论是否报错，我都会被执行。")

```

10 Python 输入与输出 (I/O) 详解

10.1 标准输入输出

- `input()`: 接收的所有内容均视为 `str`。
- `print()`: 参数 `sep` 定义间隔，`end` 定义结束符。

10.2 格式化输出 (Formatting)

Python 提供了多种字符串格式化方式。从早期传统的 `%` 号占位符，到功能丰富的 `format()`，再到目前性能最优、最简洁的 `f-string`。

10.2.1 f-string (推荐使用)

自 Python 3.6 起引入，直接在字符串前加 `f` 或 `F`，并用 `{}` 包裹表达式。它支持在格式化时进行直接运算和调用函数。

```

1 name = "Python"
2 version = 3.10
3 # 1. 基础用法
4 print(f>Welcome to {name} {version}")
5
6 # 2. 精度与填充 (常用)
7 pi = 3.1415926
8 print(f"PI is {pi:.2f}")          # 保留两位小数: 3.14
9 print(f"Value: {10:0>5}")         # 宽度为5, 右对齐, 左侧补0: 00010
10
11 # 3. 表达式运算
12 print(f"Result: {2 * 5}")         # 输出: Result: 10

```

10.2.2 `format()` 方法

通过字符串对象的 `.format()` 方法实现，具有极强的灵活性，尤其适合动态构建格式。

```

1 # 1. 位置映射
2 print("{0} is {1}, {0} is easy".format("Python", "cool"))
3
4 # 2. 关键字映射
5 print("Name: {name}, Age: {age}".format(name="Alice", age=25))
6

```

```

7 # 3. 进阶对齐与格式控制
8 # {index:[填充字符][对齐方式][宽度][.精度][类型]}
9 # 对齐方式: < 左对齐(默认), > 右对齐, ^ 居中对齐
10 text = "Hi"
11 print(f"{text:*^10}") # 输出: ****Hi**** (总宽度10, 居中, *填充)

```

10.2.3 % 占位符 (旧式)

类似于 C 语言中的 `printf`。虽然功能相对单一，但在阅读老旧代码库时经常会遇到。

- `%s` : 字符串
- `%d` : 十进制整数
- `%f` : 浮点数

```

1 data = ("List", 5)
2 print("%s has %d items" % data)
3 print("Price: %.2f" % 19.99) # 19.99

```

10.3 格式化规范详解 (Format Specification)

无论使用 `f-string` 还是 `format()`，其冒号后的语法规则是通用的：

格式化需求	语法示例	结果说明
保留小数	<code>:.3f</code>	保留 3 位小数
百分比显示	<code>:.2%</code>	<code>0.25</code> → <code>25.00%</code>
千分位分隔符	<code>,:</code>	<code>1000000</code> → <code>1,000,000</code>
进制转换	<code>:b</code> / <code>:o</code> / <code>:x</code>	转换为二进制/八进制/十六进制
科学计数法	<code>:.2e</code>	<code>1000</code> → <code>1.00e+03</code>

```

1 # 进制转换示例
2 num = 255
3 print(f"Hex: {num:x}, Oct: {num:o}, Bin: {num:b}")
4 # Hex: ff, Oct: 377, Bin: 11111111

```

10.4 文件操作 (File I/O)

Python 提供了简单而强大的文件处理机制。操作文件的基本流程为：**打开 (open)** → **操作 (read/write)** → **关闭 (close)**。

核心建议：始终使用 `with` 语句（上下文管理器），它能在操作完成后（即使发生异常）自动关闭文件，避免内存泄漏或文件句柄占用。

模式	含义	说明
'r'	只读 (Read)	默认模式。文件不存在则报错 (FileNotFoundError)。
'w'	只写 (Write)	覆盖写入。文件不存在则创建；存在则清空原有内容。
'a'	追加 (Append)	在文件末尾写入。文件不存在则创建。
'x'	创建 (Exclusive)	新建文件。如果文件已存在，则报错。
'b'	二进制 (Binary)	用于图片、视频、PDF 等非文本文件。如 'rb' 或 'wb'。
'+'	更新 (Update)	读取并写入。如 'r+' (读写), 'w+' (写读)。

10.4.1 文件打开模式详解

10.4.2 常用读取方法对比

- `f.read(n)`: 读取前 n 个字符。如果不传参数，则读取全部内容（大文件慎用）。
- `f.readline()`: 每次读取一行。
- `f.readlines()`: 读取所有行并返回一个列表。
- 迭代器 (推荐): `for line in f:` 逐行处理，对大文件最友好，内存占用极低。

10.4.3 文件指针与编码

```

1 # 指定 encoding="utf-8" 解决中文乱码问题
2 with open("note.txt", "r+", encoding="utf-8") as f:
3     content = f.read(5)    # 读取前5个字符
4     pos = f.tell()        # 获取当前文件指针位置
5     print(f"当前指针位置: {pos}")
6
7     f.seek(0)             # 将指针移动回文件开头
8     first_line = f.readline()

```

Listing 2: 指针控制与编码示例

10.4.4 综合代码示例

```

1 # 1. 写入示例
2 lines = ["Python\n", "Java\n", "C++\n"]
3 with open("langs.txt", "w", encoding="utf-8") as f:
4     f.writelines(lines) # 将列表内容写入文件
5
6 # 2. 读取与处理示例
7 try:
8     with open("langs.txt", "r", encoding="utf-8") as f:
9         # 使用 strip() 去除每行末尾的换行符
10        cleaned_data = [line.strip() for line in f if line.strip()]
11        print(f"处理后的数据: {cleaned_data}")
12 except FileNotFoundError:
13     print("错误: 文件未找到")

```


10.5 路径处理 (os.path 与 pathlib)

在实际开发中，建议配合 `os` 模块或 `pathlib` 库来处理文件路径，增强跨平台兼容性。

```
1 import os
2 # 获取绝对路径
3 abs_path = os.path.abspath("data.txt")
4 # 检查文件是否存在
5 exists = os.path.exists("data.txt")
```

11 面向对象编程 (OOP)

Python 是一门“万物皆对象”的语言。类 (Class) 是创建对象的模板，而对象 (Object) 是类的实例。

11.1 类与对象的创建

使用 `class` 关键字定义类。每个类通常包含一个初始化方法 `__init__`，用于定义实例属性。

```
1 class Animal:
2     # 类属性（所有实例共享）
3     species = "哺乳动物"
4
5     # 初始化方法（构造函数）
6     def __init__(self, name, age):
7         self.name = name # 实例属性
8         self.age = age   # 实例属性
9
10    # 实例方法
11    def say_hi(self):
12        print(f"你好，我是{self.name}")
```

11.2 实例化对象

实例化是指通过类创建具体对象的过程。

```
1 dog = Animal("旺财", 3)
2 print(dog.name)      # 访问属性
3 dog.say_hi()         # 调用方法
```

11.3 继承 (Inheritance)

继承允许我们定义一个类（子类）来继承另一个类（父类）的属性和方法，从而实现代码重用。

11.3.1 单级继承

子类继承自一个父类，并可以使用 `super()` 调用父类的方法。

```

1 class Dog(Animal):
2     def __init__(self, name, age, breed):
3         # 调用父类的初始化方法
4         super().__init__(name, age)
5         self.breed = breed
6
7     def bark(self):
8         print("汪汪!")

```

11.3.2 多级继承

子类继承自一个已经继承了其他类的子类，形成一个继承链。

```

1 class WorkingDog(Dog):
2     def work(self):
3         print(f"{self.name} 正在执勤...")

```

11.4 多态 (Polymorphism)

多态是指不同的子类对象调用相同的父类方法时，产生不同的执行结果。Python 是一种动态语言，支持“鸭子类型” (Duck Typing)。

```

1 class Cat(Animal):
2     def say_hi(self):
3         print(f"{self.name} 喵喵叫~")
4
5 # 多态体现：同样的接口，不同的实现
6 def animal_shout(obj):
7     obj.say_hi()
8
9 dog = Dog("小黑", 2, "拉布拉多")
10 cat = Cat("小花", 1)
11
12 animal_shout(dog) # 输出：你好，我是小黑
13 animal_shout(cat) # 输出：小花 喵喵叫~

```

11.5 封装与私有化 (Encapsulation)

在 Python 中，通过在属性名前加双下划线 __ 可以将其定义为私有属性，防止外部直接访问。

```

1 class BankAccount:
2     def __init__(self, balance):
3         self.__balance = balance # 私有属性
4
5     def get_balance(self):
6         return self.__balance

```

```

1 class Student:
2     def __init__(self, name, score):
3         self.name = name      # 属性
4         self.score = score
5
6     def display(self):        # 方法
7         print(f"学生: {self.name}, 分数: {self.score}")
8
9 s1 = Student("张三", 85)
10 s1.display()

```

12 常用标准库模块

- os: 操作系统接口（文件路径、环境变量）。
- sys: 系统特定参数（命令行参数 `sys.argv`）。
- datetime: 日期和时间处理。
- random: 生成随机数。
- math: 数学运算。
- json: JSON 数据解析与序列化。

```

1 import random
2 import math
3
4 print(random.randint(1, 100)) # 产生1到100的随机整数
5 print(math.sqrt(16))          # 4.0

```

Listing 3: 模块使用示例

13 总结与附录

13.1 常用内置函数表

函数	描述
<code>type(obj)</code>	返回对象的类型
<code>id(obj)</code>	返回对象的唯一内存地址
<code>isinstance(o, t)</code>	判断对象是否属于某种类型
<code>dir(obj)</code>	列出对象的所有属性和方法
<code>help(obj)</code>	显示对象的帮助文档
<code>range()</code>	生成不可变的数值序列
<code>enumerate()</code>	返回索引和值的对（常用于循环）

zip()	将多个迭代器打包成元组序列
map()	对迭代器中的每个元素执行函数
filter()	过滤序列

14 编程训练任务

1、时间计算

接收一个时间，返回该时间的下一秒。

```

1      # 输入处理
2      a = input("请输入时间 (格式 HH:MM:SS): ")
3      h, m, s = map(int, a.split(":"))
4
5      def next_sec(h, m, s):
6          s += 1
7          if s >= 60:
8              s = 0
9              m += 1
10             if m >= 60:
11                 m = 0
12                 h += 1
13                 if h >= 24:
14                     h = 0
15             return h, m, s
16
17     h, m, s = next_sec(h, m, s)
18     # 使用 zfill(2) 或格式化字符串保证两位显示
19     print(f"下一秒是: {h:02d}:{m:02d}:{s:02d}")
20

```

Listing 4: 时间计算代码

2、日期转换

输出日期是一年中的第几天。

```

1      from datetime import date
2
3      def day_of_year():
4          y = int(input("年: "))
5          m = int(input("月: "))
6          d = int(input("日: "))
7          target_date = date(y, m, d)
8          # %j 表示一年中的第几天 (001-366)
9          return int(target_date.strftime("%j"))
10
11     print(f"这是该年的第 {day_of_year()} 天")
12

```

Listing 5: 日期转换代码

3、字符串分类排序

数字与字母分类提取并排序。

```
1 string = input("请输入字符数字混合串: ")
2 digits = sorted([i for i in string if i.isdigit()])
3 alphas = sorted([i for i in string if i.isalpha()])
4
5 res_alpha = "".join(alphas)
6 res_digit = "".join(digits)
7 print(f"字母串: {res_alpha}, 数字串: {res_digit}, 合并: {res_alpha + res_digit}")
8
```

Listing 6: 字符串分类排序代码

4、均值筛选

输出超过均值的数据。

```
1 data = input("请输入整数（空格分隔）: ")
2 nums = [int(x) for x in data.split()]
3 if nums:
4     avg = sum(nums) / len(nums)
5     result = [str(x) for x in nums if x > avg]
6     print("超过均值的数据:", " ".join(result))
7
```

Listing 7: 均值筛选代码

5、验证码生成

生成 6 位大写字母与数字混合码。

```
1 import random
2 import string
3
4 def generate_code():
5     # 准备字符池: A-Z + 0-9
6     pool = string.ascii_uppercase + string.digits
7     return "".join(random.choices(pool, k=6))
8
9 print(f"随机验证码: {generate_code()}")
10
```

Listing 8: 验证码生成代码

6、素数查找

500 以内素数，每行 10 个，右对齐。

```
1 def is_prime(n):
2     if n < 2: return False
3     for i in range(2, int(n**0.5) + 1):
4         if n % i == 0: return False
```

```

5         return True
6
7     primes = [x for x in range(2, 501) if is_prime(x)]
8     for i, p in enumerate(primes):
9         print(f"{p:>4}", end=" ")
10        if (i + 1) % 10 == 0: print() # 每10个换行
11

```

Listing 9: 素数查找代码

7、递归逆序

利用递归思想翻转字符串。

```

1     def reverse_recursive(s):
2         if len(s) <= 1: return s
3         return reverse_recursive(s[1:]) + s[0]
4
5     print(reverse_recursive("Python")) # nohtyP
6

```

Listing 10: 递归逆序代码

15 经典算法

15.1 查找

15.1.1 顺序查找

逐个遍历列表，查找目标元素。复杂度 $O(n)$ 。

15.1.2 二分查找 (Binary Search)

前提: ** 必须是有序序列 **。通过折半缩小范围，复杂度 $O(\log n)$ 。

15.1.3 哈希查找 (Hash Search)

哈希查找是利用哈希表（在 Python 中体现为 `dict` 或 `set`）进行查找的方法。其理论查找时间复杂度为 $O(1)$ ，是效率最高的查找方式。

```

1 # 查找元素是否存在
2 data = [10, 25, 48, 32, 11]
3 hash_table = {val: True for val in data} # 预处理成字典
4
5 target = 32
6 if target in hash_table:
7     print(f"找到 {target}")

```

Listing 11: 哈希查找示例

15.1.4 插值查找 (Interpolation Search)

插值查找是对二分查找的改进。对于 ** 分布均匀 ** 的有序序列，它不是从中间（1/2）开始找，而是根据目标值的大小占整个范围的比例来计算预测位置。

公式： $mid = low + \frac{key - arr[low]}{arr[high] - arr[low]} \times (high - low)$

```
1 def interpolation_search(arr, key):
2     low, high = 0, len(arr) - 1
3     while low <= high and key >= arr[low] and key <= arr[high]:
4         # 核心：自适应计算 mid
5         mid = low + int((key - arr[low]) * (high - low) / (arr[high] - arr[low]))
6
7         if arr[mid] == key:
8             return mid
9         elif arr[mid] < key:
10            low = mid + 1
11        else:
12            high = mid - 1
13    return -1
```

Listing 12: 插值查找核心逻辑

15.2 查找算法对比总结

查找算法	平均复杂度	前提条件	特点
顺序查找	$O(n)$	无	简单但效率低
二分查找	$O(\log n)$	有序序列	性能稳定，最常用
插值查找	$O(\log(\log n))$	有序且均匀分布	针对特定分布效率极高
哈希查找	$O(1)$	需额外存储空间	空间换时间，速度最快

15.3 排序

15.3.1 选择排序 (Selection Sort)

每次从未排序部分选出最小的元素，放到已排序部分的末尾。

```
1 A = [49, 39, 66, 98, 75, 12, 28, 49, 56, 4]
2 for i in range(len(A)):
3     min_idx = i
4     for j in range(i + 1, len(A)):
5         if A[j] < A[min_idx]:
6             min_idx = j
7     A[i], A[min_idx] = A[min_idx], A[i]
```

15.3.2 冒泡排序 (Bubble Sort)

通过相邻元素的比较和交换，使较大的元素如同“气泡”一样逐渐移向序列末尾。

```

1 from random import randint
2 lst = [randint(1, 100) for _ in range(20)]
3 n = len(lst)
4
5 # 外层循环控制趟数
6 for i in range(n - 1):
7     # 内层循环进行相邻比较
8     for j in range(n - 1 - i):
9         if lst[j] > lst[j + 1]:
10             lst[j], lst[j + 1] = lst[j + 1], lst[j]
11
12 print("排序后结果:\n", lst)

```

Listing 13: 冒泡排序代码

15.3.3 快速排序 (Quick Sort)

快速排序通过选择一个“基准” (pivot)，将列表分为两部分：一部分比基准小，另一部分比基准大，然后递归地对这两部分进行排序。

```

1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     else:
5         pivot = arr[len(arr) // 2] # 选择中间元素作为基准
6         left = [x for x in arr if x < pivot] # 比基准小的部分
7         middle = [x for x in arr if x == pivot] # 等于基准的部分
8         right = [x for x in arr if x > pivot] # 比基准大的部分
9         return quick_sort(left) + middle + quick_sort(right)
10
11 lst = [34, 7, 23, 32, 5, 62]
12 print("快排结果:", quick_sort(lst))

```

Listing 14: 快速排序代码 (递归版)

15.3.4 插入排序 (Insertion Sort)

插入排序的工作原理是通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。类似于整理扑克牌。

```

1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5         # 将比 key 大的元素向后移动一位
6         while j >= 0 and key < arr[j]:
7             arr[j + 1] = arr[j]
8             j -= 1
9         arr[j + 1] = key
10    return arr

```



```

11
12 lst = [12, 11, 13, 5, 6]
13 print("插入排序结果:", insertion_sort(lst))

```

Listing 15: 插入排序代码

15.4 常用算法复杂度对比

在笔记中记录复杂度可以帮助你更好地理解不同场景下该选哪种算法。

算法	平均时间复杂度	最坏时间复杂度	空间复杂度
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$
选择排序	$O(n^2)$	$O(n^2)$	$O(1)$
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$
二分查找	$O(\log n)$	$O(\log n)$	$O(1)$

15.5 Python 内置排序 (Timsort)

在实际编程中，我们通常直接使用 Python 内置的方法，它们经过了高度优化。

- `list.sort()`: 原地修改列表，不返回值。
- `sorted(list)`: 返回一个新的排序列表，原列表不变。

```

1 data = [5, 2, 9, 1]
2 # 从大到小排序
3 data.sort(reverse=True)
4 print(data) # [9, 5, 2, 1]

```